

P0331r0 : Motivation and Examples for Multidimensional Array

Project: ISO JTC1/SC22/WG21: Programming Language C++
Number: P0331r0
Date: 2016-05-27
Reply-to: hcedwar@sandia.gov, balelbach@lbl.gov
Author: H. Carter Edwards
Contact: hcedwar@sandia.gov
Author: Bryce Lebach
Contact: balelbach@lbl.gov
Author: Christian Trott
Contact: crtrott@sandia.gov
Author: Mauro Bianco
Contact: mbianco@cscs.ch
Author: Robin Maffeo
Contact: Robin.Maffeo@amd.com
Author: Ben Sander
Contact: ben.sander@amd.com
Audience: Library Evolution Working Group (LEWG)
URL: https://github.com/kokkos/array_ref/blob/master/proposals/P0331.rst

Revision History	
P0009r0	Original multidimensional array reference paper with motivation, specification, and examples.
P0009r1	Revised with renaming from <code>view</code> to <code>array_ref</code> and allow unbounded rank through variadic arguments.
P03310 (<i>current</i>)	Multidimensional array reference motivation and examples moved from P0009.
References	
P0009r2	Multidimensional array reference specification
P0332	Relaxed array declaration
P0122	<code>span</code> : bounds-safe views for sequences of objects
earlier related papers: N4512, N4355, N4300, N4222	

1 Motivation for polymorphic multidimensional array reference

Multidimensional arrays are a foundational data structure for science and engineering codes, as demonstrated by their extensive use in FORTRAN for five decades. A *multidimensional array reference* is a reference to a memory extent through a **layout** mapping from a multi-index space (domain) to that extent (range). A array layout mapping may be bijective as in the case of a traditional multidimensional array, injective as in the case of a subarray, or surjective to express symmetry.

Traditional layout mappings have been specified as part of the language. For example, FORTRAN specifies *column major* layout and C specifies *row major* layout. Such a language-imposed specification requires significant code refactoring to change an array's layout, and requires significant code complexity to implement non-traditional layouts such as tiling in modern linear algebra or structured grid application domains. Such layout changes are required to adapt and optimize code for varying computer architectures; for example, to change a code from *array of structures* or *row major* storage to *structure of arrays* or *column major* storage. Furthermore, multiple versions of code must be maintained for each required layout.

A multidimensional array reference abstraction with a polymorphic layout is required to enable changing array layouts without extensive code refactoring and maintenance of functionally redundant code. Layout polymorphism is a critical capability; however, it is not the only beneficial form of polymorphism.

The Kokkos library (github.com/kokkos/kokkos) implements multidimensional array references with polymorphic layout, and other access properties as well. Until recently the Kokkos implementation was limited to C++1998 standard and is incrementally

being refactored to C++2011 standard. Additionally, there is a standalone reference implementation of this proposal which is publicly available on github (github.com/kokkos/array_ref).

2 Motivation for Syntax

The full `array_ref` library specification is given in P0009r2.

```
namespace std {
namespace experimental {
    template< typename DataType , typename ... Properties >
    struct array_ref ;
}}
```

2.1 One-Dimensional Array

A reference to a one-dimension array is anticipated to subsume the functionality of a pointer to a memory extent combined with an array length. For example, a one-dimensional array is passed to a function as follows.

```
void foo( int A[] , size_t N ); // Traditional API
void foo( const int A[] , size_t N ); // Traditional API

void foo( array_ref< int[] > A ); // Reference API
void foo( array_ref< const int[] > A ); // Reference API

void bar()
{
    enum { L = ... };
    int buffer[ L ];
    array_ref<int[]> A( buffer , L );

    assert( L == A.size() );
    assert( & A[0] == buffer );

    foo( array );
}
```

The *const-ness* of an `array_ref` is analogous to the *const-ness* of a pointer. A `const array_ref<D>` is similar to a const-pointer in that the `array_ref` may not be modified but the referenced extent of memory may be modified. A `array_ref<const D>` is similar to a pointer-to-const in that the referenced extent of memory may not be modified. These are the same *const-ness* semantics of `unique_ptr` and `shared_ptr`.

The `T[]` syntax has precedence in the standard; `unique_ptr` supports this syntax to denote a `unique_ptr` which manages the lifetime of a dynamically allocated array of objects.

2.2 Traditional Multidimensional Array with Static Dimensions

A traditional multidimensional array with static dimensions (for example, an array of 3x3 tensors) is passed to a function as follows.

```
void foo( double A[][3][3] , size_t N0 ); // Traditional API
void foo( array_ref< double[][3][3] > A ); // Reference API

void bar()
{
    enum { L = ... };
    int buffer[ L * 3 * 3 ];
    array_ref< double[][3][3] > A( buffer , L );

    assert( 3 == A.rank() );
    assert( L == A.extent(0) );
    assert( 3 == A.extent(1) );
    assert( 3 == A.extent(2) );
    assert( A.size() == A.extent(0) * A.extent(1) * A.extent(2) );
    assert( & A(0,0,0) == buffer );

    foo( A );
}
```

Support for static extents is an essential performance feature of the proposed `array_ref`. First a compiler can optimize the index-to-object mapping computation and second the `array_ref` implementation can eliminate storage for static extents. Consider the following example where `L` is only known at runtime.

The member access mapping can be optimized to a single integer multiply-add `A.ptr[i*9+7]` because the implementation of `A(i,2,1)` is `A.ptr[((i)*3+j)*3+k]` and `j=2` and `k==1` are statically determined. The `sizeof(A)` can be `sizeof(double*)+sizeof(size_t)` because storage is required for only the pointer and dynamic extent.

2.3 Multidimensional Array with Multiple Dynamic Dimensions

The current multidimensional array type declaration in **n4567 8.3.4.p3** restricts array declarations such that only the leading dimension may be implicit. Multidimensional arrays with multiple implicit (dynamic) dimensions as well as static dimensions are supported with the **dimension** property. The dimension property uses the "magic value" zero to denote an implicit dimension. The "magic value" of zero is chosen for consistency with **std::extent**.

```
array_ref< int[][3] > x ;

assert( x.extent(0) == 0 );
assert( x.extent(1) == 3 );

assert( extent< int[][3] , 0 >::value == 0 );
assert( extent< int[][3] , 1 >::value == 0 );

array_ref< int , array_property::dimension<0,0,3> > y ;
assert( y.extent(0) == 0 );
assert( y.extent(1) == 0 );
assert( y.extent(2) == 3 );

array_ref< int , array_property::dimension<0,0,3> > z(ptr,N0,N1);
assert( z.extent(0) == N0 );
assert( z.extent(1) == N1 );
assert( z.extent(2) == 3 );
```

2.4 Preferred Syntax

We prefer the following concise and intuitive syntax for arrays with multiple implicit dimensions.

```
array_ref< int[][][3] > y ; // concise intuitive syntax
```

However, this syntax requires a relaxation of the current multidimensional array type declaration in **n4567 8.3.4.p3**, as proposed in P00332. Furthermore, this concise and intuitive syntax eliminates the need for `array_property::dimension<...>` and the associated "magic value" of zero to denote an implicit dimension.

3 Array Properties

3.1 Layout Polymorphism

The `array_ref::operator()` maps the input multi-index from the array's cartesian product multi-index *domain* space to a member in the array's *range* space. This is the **layout** mapping for the referenced array. For natively declared multidimensional arrays the layout mapping is defined to conform to treating the multidimensional array as an *array of arrays of arrays ...*; i.e., the size and span are equal and the strides increase from right-to-left (the layout specified in the C language). In the FORTRAN language defines layout mapping with strides increasing from left-to-right. These *native* layout mappings are only two of many possible layouts. For example, the *basic linear algebra subprograms (BLAS)* standard defines dense matrix layout mapping with padding of the leading dimension, requiring both dimensions and **LDA** parameters to fully declare a matrix layout.

A property template parameter specifies a layout mapping. If this property is omitted the layout mapping of the array reference conforms to a corresponding natively declared multidimensional array as if implicit dimensions were declared explicitly. The default layout is *regular* - the distance is constant between entries when a single index of the multi-index is incremented. This distance is the *stride* of the corresponding dimension. The default layout mapping is bijective and the stride increases monotonically from the right most to the left most dimension.

```
// The default layout mapping of a rank-four multidimensional
// array is as if implemented as follows.

template< size_t N0 , size_t N1 , size_t N2 , size_t N3 >
size_t native_mapping( size_t i0 , size_t i1 , size_t i2 , size_t i3 )
{
    return i0 * N3 * N2 * N1 // stride == N3 * N2 * N1
        + i1 * N3 * N2      // stride == N3 * N2
        + i2 * N3           // stride == N3
        + i3 ;              // stride == 1
}
```

An initial set of layout properties are **layout_right**, **layout_left**, and **layout_stride**,

```
namespace std {
namespace experimental {
namespace array_property {
    struct layout_right ;
    struct layout_left ;
    struct layout_stride ;
}}}

```

A **void** (*a.k.a.*, default or native) mapping is regular and bijective with strides increasing from increasing from right most to left most dimension. A **layout_right** mapping is regular and injective (may have padding) with strides increasing from right most to left most dimension. A **layout_left** mapping is regular and injective (may have padding) with strides increasing from left most to right most dimension. A **layout_stride** mapping is regular; however, it might not be injective or surjective.

```
// The right and left layout mapping of a rank-four
// multidimensional array could be is as if implemented
// as follows. Note that padding is allowed but not required.

template< size_t N0 , size_t N1 , size_t N2 , size_t N4 >
size_t right_padded_mapping( size_t i0 , size_t i1 , size_t i2 , size_t i3 )
{
    const size_t S3 = // stride of dimension 3
    const size_t P3 = // padding of dimension 3
    const size_t P2 = // padding of dimension 2
    const size_t P1 = // padding of dimension 1
    return i0 * S3 * ( P3 + N3 ) * ( P2 + N2 ) * ( P1 + N1 )
        + i1 * S3 * ( P3 + N3 ) * ( P2 + N2 )
        + i2 * S3 * ( P3 + N3 )
        + i3 * S3 ;
}

template< size_t N0 , size_t N1 , size_t N2 , size_t N4 >
size_t left_padded_mapping( size_t i0 , size_t i1 , size_t i2 , size_t i3 )
{
    const size_t S0 = // stride of dimension 0
    const size_t P0 = // padding of dimension 0
    const size_t P1 = // padding of dimension 1
    const size_t P2 = // padding of dimension 2
    return i0 * S0
        + i1 * S0 * ( P0 + N0 )
        + i2 * S0 * ( P0 + N0 ) * ( P1 + N1 )
        + i3 * S0 * ( P0 + N0 ) * ( P1 + N1 ) * ( P2 + N2 );
}

```

3.2 Extensible Layout Polymorphism

The `array_ref` is intended to be extensible such that a user may supply a customized layout mapping. A user supplied customized layout mapping will be required to conform to a specified interface; *a.k.a.*, a C++ Concept. Details of this extension point will be included in a subsequent proposal. Our current extensibility strategy is for a user supplied layout property to implement an offset mapping.

Motivation: An important customized layout mapping is hierarchical tiling. This kind of layout mapping is used in dense linear algebra matrices and computations on Cartesian grids to improve the spatial locality of array entries. These mappings are bijective but are not regular. Computations on such multidimensional arrays typically iterate through tiles as *subarray* of the array.

```
template< size_t N0 , size_t N1 , size_t N2 >
size_t tiling_left_mapping( size_t i0 , size_t i1 , size_t i2 )
{
    static constexpr size_t T = // cube tile size
    constexpr size_t T0 = ( N0 + T - 1 ) / T ; // tiles in dimension 0
    constexpr size_t T1 = ( N1 + T - 1 ) / T ; // tiles in dimension 1
    constexpr size_t T2 = ( N2 + T - 1 ) / T ; // tiles in dimension 2

    // offset within tile + offset to tile
    return ( i0 % T ) + T * ( i1 % T ) + T * T * ( i2 % T )
        + T * T * T * ( ( i0 / T ) + T0 * ( ( i1 / T ) + T1 * ( i2 / T ) ) );
}

```

Note that a tiled layout mapping is irregular and if padding is required to align with tile boundaries then the span will exceed the size. A customized layout mapping will have slightly different requirements depending on whether the layout is regular or

irregular.

3.3 Bounds Checking

Array bounds checking is an invaluable tool for debugging user code. This functionality traditionally requires global injection through special compiler support. In large, long running code global array bounds checking introduces a significant overhead that impedes the debugging process. A member access array bounds checking array property allows the selective injection of array bounds checking and removes the need for special compiler support. A high quality implementation of bounds checking would output the array bounds, multi-index, and traceback of where the array bounds violation occurred.

```
// User enables array bounds checking for selected array_ref.

array_ref< int , array_property::dimension<0,0,3>
    , array_property::check_bounds_if< ENABLE_ARRAY_BOUNDS_CHECKING > >
    x(ptr,N0,N1);
```

3.4 Future Possible Extensions

The `array_ref` abstraction and interface has utility well beyond the multidimensional array layout property. Other planned and prototyped properties include specification of which *memory space* within a heterogeneous memory system the referenced data resides on and algorithmic access intent properties. Examples of access intent properties include

1. *read-only random with locality* such that member queries are performed through GPU texture cache hardware for GPU memory spaces,
2. *atomic* such that member access operations are overloaded via proxy objects to atomic operations (see P0019, Atomic View),
3. *non-temporal* such that member access operations can be overloaded with non-caching reads and writes, and
4. *restrict* to guarantee non-aliasing of referenced data within the current context.

4 Subarrays

The capability to **easily** extract subarrays of an array, or subarrays of subarrays, is essential to many array-based algorithms.

```
using U = array_ref< int , array_properties::dimension<0,0,0> > ;

U x(buffer,N0,N1,N2);

// Using std::pair<int,int> for an integral range
auto y = subarray( x , std::pair<int,int>(1,N0-1) ,
                  std::pair<int,int>(1,N1-1) , 1 );

assert( y.rank() == 2 );
assert( y.extent(0) == N0 - 2 );
assert( y.extent(0) == N1 - 2 );
assert( & y(0,0) == & x(1,1,1) );

// Using initializer_list of size 2 as an integral range
auto z = subarray( x , 1 , {1,N1-1} , 1 );

assert( z.rank() == 1 );
assert( & z(0) == & x(1,1,1) );

// Conveniently extracting subarray for all of a extent
// without having to explicitly extract the dimensions.
auto x = subarray( x , array_property::all , 1 , 1 );
```

The `subarray()` function returns an unspecified instantiation of `array_ref<>`. Note that there is precedence in the standard for library functions with unspecified return types (e.g. `bind()`).

4.1 Subarray Type Deduction

The `subarray` function returns `array_ref< deduced... >`. The return type is deduced from the input `array_ref` and the slicing argument pack. The deduction rules must be defined to insure correctness and should be defined for performance. For example, a simple rule would define the returned type to always have a strided layout. While correct there are many use cases where a better performing layout can be deduced.

Subarray type deduction is necessarily dependent upon the layout.

4.2 Example Usage in an 8th Order Finite Difference Stencil

The subarray interface provides a powerful mechanism for accessing 3-dimensional data in numerical kernels in a fashion which utilizes performant memory access patterns and is amenable to compiler-assisted vectorization.

The following code is an example of a typical finite difference stencil which might be used in a computational fluid dynamics application. This code utilizes operator splitting to avoid vector register pressure and moves through memory in unit stride to facilitate optimal memory access patterns. With the addition of compiler alignment hints (as well as padding and aligned allocations to make those assumptions true) and compiler directives or attributes to indicate that the input pointers do not alias each other, this code would vectorize well on a traditional x86 platform.

```
void eighth_order_stencil(
    const double* V, double* U,
    ptrdiff_t dx, ptrdiff_t dy, ptrdiff_t dz,
    array<double, 5> c)
{
    // Iterate over interior points, skipping the 4 cell wide ghost
    // zone region.
    for (int iz = 4; iz < dz - 4; ++iz)
        for (int iy = 4; iy < dy - 4; ++iy) {
            // Pre-compute shared iy and iz indexing to ensure redundant
            // calculations are avoided.
            double const* v = &V[iy*dx + iz*dx*dy];
            double* u = &U[iy*dx + iz*dx*dy];

            // X-direction (unit stride) split.
            for (int ix = 4; ix < dx - 4; ++ix)
                u[ix] = c[0] * v[ix]
                    + c[1] * (v[ix+1] + v[ix-1])
                    + c[2] * (v[ix+2] + v[ix-2])
                    + c[3] * (v[ix+3] + v[ix-3])
                    + c[4] * (v[ix+4] + v[ix-4]);

            // Y-direction (dx stride) split.
            for (int ix = 4; ix < dx - 4; ++ix)
                u[ix] += c[1] * (v[ix+dx] + v[ix-dx])
                    + c[2] * (v[ix+2*dx] + v[ix-2*dx])
                    + c[3] * (v[ix+3*dx] + v[ix-3*dx])
                    + c[4] * (v[ix+4*dx] + v[ix-4*dx]);

            // Z-direction (dx*dy stride) split.
            for (int ix = 4; ix < dx - 4; ++ix)
                u[ix] += c1 * (v[ix+dx*dy] + v[ix-dx*dy])
                    + c2 * (v[ix+2*dx*dy] + v[ix-2*dx*dy])
                    + c3 * (v[ix+3*dx*dy] + v[ix-3*dx*dy])
                    + c4 * (v[ix+4*dx*dy] + v[ix-4*dx*dy]);
        }
}
```

The corresponding code can be rewritten using `array_ref<>` and the associated `subarray()` interfaces. Note that all the code is now decoupled from the arrays' layout.

```
template< typename ... VP , typename ... UP >
void eighth_order_stencil(
    array_ref<const double, array_property::dimension<0, 0, 0>, VP... > const V,
    array_ref<double, array_property::dimension<0, 0, 0>, UP... > const U,
    array<double, 5> const c)
{
    auto all = array_property::all ;

    const int base = 4 ;
    const int endx = U.extent(0) - base ;
    const int endy = U.extent(1) - base ;
    const int endz = U.extent(2) - base ;

    for ( int iz = base ; iz < endz ; ++iz )
        for ( int iy = base ; iy < endy ; ++iy ) {

            // Use subarrays to avoid redundant indexing calculations
            // within the inner loop.

            auto u = subarray( U, all, iy, iz);
            auto vx = subarray( V, all, iy, iz);
            auto vy = subarray( V, all , {iy-base,iy+base+1}, iz );
            auto vz = subarray( V, all , iy, {iz-base,iz+base+1} );
```

```

// X-direction split.
for (int ix = base ; ix < endx ; ++ix)
    u[ix] = c[0] * vx[ix]
        + c[1] * ( vx[ix+1] + vx[ix-1] )
        + c[2] * ( vx[ix+2] + vx[ix-2] )
        + c[3] * ( vx[ix+3] + vx[ix-3] )
        + c[4] * ( vx[ix+4] + vx[ix-4] );

// Y-direction split.
for (int ix = base ; ix < endx ; ++ix)
    u[ix] += c[1] * ( vy(ix,base+1) + vy(ix,base-1) )
        + c[2] * ( vy(ix,base+2) + vy(ix,base-2) )
        + c[3] * ( vy(ix,base+3) + vy(ix,base-3) )
        + c[4] * ( vy(ix,base+4) + vy(ix,base-4) );

// Z-direction split.
for (int ix = base ; ix < endx ; ++ix)
    u[ix] += c[1] * ( vz(ix,base+1) + vz(ix,base-1) )
        + c[2] * ( vz(ix,base+2) + vz(ix,base-2) )
        + c[3] * ( vz(ix,base+3) + vz(ix,base-3) )
        + c[4] * ( vz(ix,base+4) + vz(ix,base-4) );

}
}

```